

1. உன் ecommerce project architecture explain பண்ணு.

என் ecommerce project ஒரு API-based architecture follow பண்ணி build பண்ணிருக்கேன். இதில் frontend side-ல HTML, CSS, Bootstrap மற்றும் JavaScript use பண்ணிருக்கேன். Backend side-ல PHP மற்றும் MySQL use பண்ணி முழு business logic handle பண்ணிருக்கேன். Frontend நேரடியாக database-ஐ access பண்ணாது. அதற்கு பதிலாக backend APIs மூலம் data exchange நடக்கும்.

Guest user case-ல cart items localStorage-ல store ஆகும். User login ஆனதும் அந்த localStorage data backend API மூலம் database-க்கு sync ஆகும். Logged-in users direct-ஆ database-ல cart and order operations பண்ணுவாங்க.

2. API-based design ஏன் choose பண்ணின?

API-based design choose பண்ண காரணம் frontend மற்றும் backend completely separate ஆக maintain பண்ண முடியும் என்பதுதான். இது project-ஐ scalable ஆகவும் maintainable ஆகவும் ஆக்கும்.

Future-ல அதே backend APIs-ஐ mobile app அல்லது React/Angular frontend-க்கு reuse பண்ண முடியும். So one backend, multiple frontends support பண்ண முடியும் என்பது பெரிய advantage.

3. Frontend & backend separation எப்படி achieve பண்ணின?

Frontend மற்றும் backend strict separation maintain பண்ணிருக்கேன். Frontend-ல UI மட்டும் இருக்கும், data processing எதுவும் இருக்காது. JavaScript மூலம் fetch or AJAX பயன்படுத்தி backend APIs-க்கு request அனுப்பப்படும். Backend-ல PHP files REST APIs மாதிரி work பண்ணி database operations handle பண்ணும்.

இதனால் UI changes and backend logic independent-ஆ work ஆகும், one side modify பண்ணினாலும் மற்ற side affect ஆகாது.

4. Projectல main modules என்னென்ன?

இந்த project-ல main modules என்று சொன்னா user module, product module, cart module, order module மற்றும் admin module இருக்கிறது. User module-ல registration மற்றும் login functionality இருக்கும். Product module-ல products list, view மற்றும் search handle பண்ணப்படும். Cart module-ல guest cart localStorage-லவும் logged-in cart database-லவும் manage பண்ணப்படும். Order module checkout process மற்றும் order history handle பண்ணும். Admin module மூலம் products, categories மற்றும் users manage பண்ண முடியும்.

5. User login இல்லாம cart items localStorageல store பண்ணுற logic explain பண்ணு.

User login இல்லாத போது cart items அனைத்தும் JavaScript மூலம் handle பண்ணப்படும். User “Add to Cart” click பண்ணினதும் அந்த product details ஒரு object ஆக create பண்ணி, அதை localStorage-ல JSON format-ல store பண்ணுவேன். ஒவ்வொரு time add பண்ணும் போதும் existing localStorage data fetch பண்ணி அதில் new product append பண்ணி மீண்டும் set பண்ணுவேன். இதனால் page refresh ஆனாலும் cart data retain ஆகும்.

6. Login ஆனதும் localStorage cart data database-க்கு எப்படி sync பண்ணின?

User login success ஆனதும் frontend-ல localStorage cart data fetch பண்ணுவேன். அந்த data-வை API request மூலம் backend-க்கு அனுப்புவேன். Backend PHP side-ல அந்த cart items loop பண்ணி user_id உடன் cart table-ல insert பண்ணும். அதே சமயம் already DB-ல இருக்கும் cart items இருந்தால் update logic apply பண்ணப்படும். Sync முடிந்த பிறகு localStorage clear பண்ணிடுவேன், இதனால் duplicate data avoid ஆகும்.

7. Duplicate products வந்தா எப்படி handle பண்ணின?

Duplicate products வந்தால் backend-ல check பண்ணுவேன். அதாவது same user_id மற்றும் product_id already cart table-ல இருந்தால் புதிய row create பண்ணாமல் quantity மட்டும் increment பண்ணுவேன். இதனால் database clean-ஆவும் consistent-ஆவும் இருக்கும்.

8. Guest cart vs logged-in cart conflict வந்தா எப்படி solve பண்ணின?

Guest cart மற்றும் logged-in cart merge செய்யும் போது இரண்டு sources-ல இருந்தும் cart data fetch பண்ணுவேன். localStorage cart மற்றும் database cart இரண்டையும் compare பண்ணி same product இருந்தால் quantity merge பண்ணுவேன். இல்லையெனில் separate items ஆக add பண்ணுவேன். இதனால் user எந்த device அல்லது state-ல இருந்தாலும் final cart consistent ஆக இருக்கும்.

10. Cart merge logic backendல பண்ணினியா frontendல பண்ணினியா? ஏன்?

Cart merge logic mostly backend-ல தான் handle பண்ணினேன். காரணம் backend தான் single source of truth ஆக இருக்கும். Frontend-ல merge பண்ணினா tampering அல்லது inconsistency வர வாய்ப்பு இருக்கும். Backend-ல handle பண்ணினா data secure-ஆவும் reliable-ஆவும் இருக்கும். Frontend மட்டும் data send பண்ணும் role-ல இருக்கும், actual merge and validation backend-ல நடக்கும்.

11. Login system எப்படி implement பண்ணின?

Login system PHP backend மூலம் implement பண்ணிருக்கேன். User login form-ல email மற்றும் password enter பண்ணினதும் அந்த data backend API-க்கு send ஆகும். Backend-ல MySQL database-ல அந்த email check பண்ணி, password hash verify பண்ணுவேன். Credentials correct இருந்தால் user-க்கு session create பண்ணி login success response frontend-க்கு return பண்ணுவேன். Frontend அந்த response based-ஆ user-ஐ home page அல்லது dashboard-க்கு redirect பண்ணும்.

12 Session உன் projectல எது use பண்ணின? ஏன்?

இந்த project-ல நான் PHP Session தான் use பண்ணினேன். காரணம் இது traditional web-based ecommerce project, மேலும் PHP session server-side-ல maintain ஆகும் என்பதால் implementation simple மற்றும் secure ஆக இருக்கும்.

13. Login success ஆனதும் user state எப்படி maintain பண்ணின?

Login success ஆனதும் backend PHP side-ல session create பண்ணுவேன். அதாவது user_id, username, role போன்ற details store பண்ணப்படும். இந்த session-ஐ PHP itself internally handle பண்ணும். Session ID automatically browser-க்கு pass ஆகும்

Frontend side-ல ஒவ்வொரு page load அல்லது API call போதும் “check session” API call பண்ணுவேன். Backend அந்த session valid-ஆ இருக்கா என்று verify பண்ணி, user logged-in state return பண்ணும். அதைப் base பண்ணி frontend UI update பண்ணுவேன்.

இதனால் page refresh ஆனாலும் user session server-side-ல maintain ஆகும், user manually logout பண்ணும் வரை state lost ஆகாது.

14. Security issues (like token misuse) எப்படி prevent பண்ணின?

Security prevent பண்ண நான் சில measures follow பண்ணினேன். Passwords database-ல plain text-ஆ store பண்ணாமல் hashing (bcrypt) use பண்ணினேன். Session hijacking avoid பண்ண session regeneration login time-ல பண்ணினேன். Each API request-க்கும் session validation mandatory ஆக்கினேன். மேலும் direct API access prevent பண்ண authentication check இல்லாமல் எந்த sensitive operation-மும் allow பண்ணலை.

15. Cart add API flow எப்படி work ஆகுது?

User “Add to Cart” click பண்ணினதும் frontend-ல product id, quantity போன்ற details fetch பண்ணி API call அனுப்பப்படும். Backend PHP side-ல முதலில் user logged-in ஆ இருக்கானு session check பண்ணுவேன். Logged-in user என்றால் MySQL cart table-ல அந்த product already இருக்கா என்று check பண்ணுவேன். இருந்தா quantity update பண்ணுவேன், இல்லையெனில் புதிய row insert பண்ணுவேன். Guest user என்றால் localStorage-ல store பண்ணும் logic frontend-ல handle ஆகும்.

16. API authentication check எப்படி handle பண்ணின?

API authentication mostly PHP session based-ஆ handle பண்ணினேன். Each protected API call-க்கும் session check பண்ணுவேன். Session valid இருந்தால் மட்டும் data access allow பண்ணுவேன். இல்லையெனில் unauthorized response return பண்ணுவேன். இதனால் direct API URL hit பண்ணினாலும் unauthorized user data access பண்ண முடியாது.

17. Tables structure explain பண்ணு (users, products, cart)

என் ecommerce project-ல MySQL database design logical separation follow பண்ணிருக்கேன்.

Users table-ல user details like id, name, email, password, phone no, பண்ணப்படும. Products table-ல product information like name, price, image, description, category_id, stocks இருக்கும். Cart table-ல user selected products temporary-ஆ store பண்ணப்படும், order place பண்ணும் வரைக்கும் irukum.

18. Cart table design எப்படி பண்ணின?

Cart table design simple ஆனால் scalable ஆக இருக்கும் மாதிரி பண்ணிருக்கேன். அதுல முக்கிய fields user_id, product_id, quantity இருக்கும். இதன் மூலம் each user-க்கு multiple products map ஆகும். Same product duplicate வராமல் avoid பண்ண user_id + product_id combination மூலம் unique logic maintain பண்ணுவேன். Quantity update logic மூலம் same product repeated rows avoid பண்ணப்படும்.

19. Order placement time cart data எப்படி convert ஆகுது?

User checkout பண்ணும் போது முதலில் அந்த user-க்கு related cart items database-ல இருந்து fetch பண்ணுவேன். அந்த cart data வைத்து frontend or backend side-ல total amount calculate பண்ணப்படும்.

Order concept-க்கு separate orders table maintain பண்ணல, அதனால final purchase record store பண்ணாமல், checkout success ஆனதும் அந்த cart items-ஐ process பண்ணி immediately cart-ல இருந்து remove பண்ணிடுவேன்.

அதாவது cart தான் temporary storage மாதிரி use ஆகுது. Order place ஆனதும் அந்த cart data clear பண்ணப்படும், இதனால் user-க்கு cart → empty state வரும், and purchase flow complete ஆகும்.

20. Browser change ஆனா cart data loss ஆகும் situation handle பண்ணினியா?

ஆம், localStorage browser-specific என்பதால் browser change அல்லது device change ஆனா data loss ஆகும். இதை handle பண்ண login time-ல localStorage cart data-வை backend-க்கு sync பண்ணி database-ல store பண்ணுவேன். இதனால் user login பண்ணின பிறகு எந்த device-ல இருந்தாலும் cart data retrieve பண்ண முடியும்.

21. LocalStorage → DB migration logic step-by-step explain பண்ணு.

User login success ஆனதும் frontend-ல localStorage-ல இருக்கும் cart data fetch பண்ணுவேன். அந்த data JSON format-ல backend API-க்கு send பண்ணப்படும். Backend PHP side-ல அந்த cart items loop பண்ணி each product-க்கும் user_id attach பண்ணி database cart table-ல insert அல்லது update பண்ணுவேன். Same product already இருந்தா quantity update பண்ணுவேன். Migration முடிந்த பிறகு localStorage clear பண்ணுவேன், இதனால் duplicate data avoid ஆகும் மற்றும் DB தான் final source of truth ஆகும்.

22. AJAX / fetch API use பண்ணினியா?

நான் AJAX அல்லது fetch API மூலம் real-time dynamic update implement பண்ணல. என் project-ல page reload ஆன பிறகுதான் backend-ல இருந்து products fetch பண்ணி display ஆகும்.

அதாவது product listing load ஆகும் போது full page reload அல்லது page load time-ல PHP API call மூலம் database-ல இருந்து products fetch பண்ணி frontend-ல render பண்ணப்படும்.

Cart add போன்ற சில flows localStorage use பண்ணி handle பண்ணினேன், but UI update real-time AJAX style-ல continuously refresh ஆகும் மாதிரி implement பண்ணல.

23. Bootstrap responsive design எப்படி handle பண்ணின?

Bootstrap grid system use பண்ணி responsive design achieve பண்ணினேன். Different screen sizes-க்கு col-md, col-sm போன்ற classes use பண்ணி layout adjust பண்ணினேன். Mobile-first approach follow பண்ணி small screen-ல single column layout, desktop-ல multi-column layout maintain பண்ணினேன். இதனால் same UI all devices-ல properly work ஆகும்

24. Stocks feature உன் project-ல எப்படி work ஆகுது?

என் ecommerce project-ல product stock quantity database-ல maintain பண்ணப்படும். Frontend-ல அந்த stock value based-ஆ availability status display பண்ணுவேன்.

Stock 5-க்கு மேல் இருந்தா “Available” அல்லது normal stock count show பண்ணப்படும். Stock 5 அல்லது அதற்கு கீழ் இருந்தா remaining quantity highlight பண்ணுவேன், like “Only 3 left”, “Only 2 left” என்று show பண்ணுவேன். இதனால் user-க்கு product availability தெளிவாக தெரியும் மற்றும் urgency feel create ஆகும்.

இந்த project version-ல stock deduction logic order time-ல implement பண்ணல. இது mainly UI/UX display purpose-க்காக use பண்ணப்பட்டது.

25. Future-ல stock automatic reduce ஆகும் மாதிரி plan என்ன?

Future-ல இதை full inventory system மாதிரி upgrade பண்ண plan பண்ணிருக்கேன். User order place பண்ணும் போது backend-ல stock validation பண்ணி, order quantity based-ஆ stock automatically reduce ஆகும் logic implement பண்ணுவேன்.

Checkout time-ல current stock database-ல இருந்து fetch பண்ணி check பண்ணுவேன். Available இருந்தா SQL transaction மூலம் stock = stock - ordered_quantity என்று update பண்ணுவேன். இதை proper transaction handling-ல பண்ணினா multiple users same time order பண்ணினாலும் data inconsistency avoid ஆகும்.

இதனால் system real-time ecommerce inventory flow மாதிரி behave ஆகும்

26. Stock 0 ஆகிட்டா என்ன நடக்கும்?

Stock 0 ஆனதும் அந்த product frontend-ல “Out of Stock” என்று display பண்ணப்படும். Add to cart button disable பண்ணப்படும் அல்லது hide பண்ணப்படும். இதனால் user unavailable product order பண்ண முடியாது.

27. Stock low alert implement பண்ணினியா?

Yes (if asked), low stock threshold set பண்ணலாம். Example stock < 5 இருந்தா frontend-ல “Only few left” என்று show பண்ணலாம்.

28. Stocks quantity frontend-ல எப்படி control பண்ணுறீங்க?

என் ecommerce project-ல product stock quantity database-ல இருந்து fetch பண்ணி frontend-ல enforce பண்ணிருக்கேன். User quantity increase பண்ணும் போது available stock limit check பண்ணுவேன்.

உதாரணமாக stock 10 இருந்தா, quantity selector 10-க்கு மேல increase ஆக விடமாட்டேன். User 10க்கு மேல try பண்ணினா frontend-ல immediate alert / notification show பண்ணுவேன் like “Only 10 items available in stock” என்று.

இதனால் user wrong quantity order பண்ண முடியாது மற்றும் stock limit respect பண்ணப்படும். இது frontend validation + UX improvement purpose-க்காக implement பண்ணினேன்.

29. Admin panel-ல என்ன features implement பண்ணின?

என் project-ல admin panel மூலம் ecommerce முழு management handle பண்ண முடியும். Admin login ஆனதும் products, categories மற்றும் users manage பண்ண முடியும். Product add, edit, delete பண்ணலாம். Category create பண்ணி products-க்கு assign பண்ணலாம். User list view பண்ணி basic management செய்ய முடியும்.

30. Admin product add பண்ணினா frontend-ல எப்படி reflect ஆகுது?

Admin product add பண்ணினதும் backend PHP மூலம் product database-ல store ஆகும். Frontend product listing page API call மூலம் database-ல இருந்து latest data fetch பண்ணும். அதனால் page reload அல்லது refresh ஆனதும் புதிய product automatically show ஆகும்.

31. Admin side-ல product delete/edit எப்படி handle பண்ணின?

Admin edit அல்லது delete button click பண்ணினதும் product id backend-க்கு அனுப்பப்படும். Edit case-ல existing product data update பண்ணப்படும். Delete case-ல அந்த product database-ல இருந்து remove பண்ணப்படும்.

32. Order table implement பண்ணலன்னா என்ன சொல்வது?

என் project-ல full-fledged orders table implement பண்ணல. Checkout process simple flow-ஆ design பண்ணப்பட்டது. User cart items confirm பண்ணினதும், cart data process பண்ணி cart-ல இருந்து clear பண்ணுவேன். அதையே temporary order completion மாதிரி treat பண்ணினேன்.

அதாவது cart itself temporary storage ஆக use ஆகுது. Order history maintain பண்ணும் separate table structure இந்த version-ல இல்லை

33. Cookies vs Local Storage vs Session Storage

Cookies, Local Storage, Session Storage மூன்றும் browser storage mechanisms தான், ஆனால் இவை purpose, size limit, lifetime, security எல்லாம் வேறுபடும்.

Cookies என்பது server-க்கு request உடன் automatically send ஆகும் small data storage. இதை mostly authentication, session tracking போன்றவற்றுக்கு use பண்ணுவாங்க. Cookies size குறைவு (around 4KB) மற்றும் expiry time set பண்ணலாம். ஆனால் every HTTP request-க்கும் send ஆகுதால் performance impact இருக்கலாம்.

Local Storage என்பது browser-ல permanently data store பண்ணும் mechanism. இதுக்கு expiry time இல்ல, user manually clear பண்ணும் வரை data இருக்கும். Size அதிகம் (around 5–10MB). இது server-க்கு automatically send ஆகாது. அதனால் cart data, preferences போன்றவற்றுக்கு use பண்ணலாம்.

Session Storage என்பது local storage மாதிரி தான், ஆனால் tab/session level storage. Browser tab close பண்ணினா data delete ஆகும். இது also server-க்கு send ஆகாது. Temporary data maintain பண்ண use ஆகும்.

34. JSON-ல eppadi store panniga?

என் project-ல localStorage பயன்படுத்தும் போது cart data-வை JSON format-ல தான் store பண்ணினேன். JavaScript object-ஆ இருக்குற product details-ஐ JSON.stringify() மூலம் string-ஆ convert பண்ணி localStorage-ல save பண்ணுவேன்.

மீண்டும் அந்த data use பண்ணும் போது JSON.parse() மூலம் மீண்டும் object format-க்கு convert பண்ணி cart display மற்றும் calculation பண்ணுவேன்.

இதனால் multiple products array format-ல structured-ஆ store பண்ண முடிந்தது.

35. API-ல இருந்து frontend-க்கு data எப்படி show பண்ணினீங்க?

Backend-ல PHP மூலம் REST API create பண்ணினேன். அந்த API database-ல இருந்து data fetch பண்ணி JSON format-ல return பண்ணும். Frontend side-ல JavaScript fetch() method use பண்ணி அந்த API call பண்ணுவேன்.

API response வந்ததும் அந்த JSON data-வை JavaScript-ல parse பண்ணி, loop மூலம் HTML elements create பண்ணுவேன். பிறகு அந்த elements DOM-ல append பண்ணி product list அல்லது cart UI-யா display பண்ணுவேன்.